

GitHub Copilot CLI

Security & Developer Training Guide

Audience: Security, DevOps, and Engineering leaders rolling out GitHub Copilot CLI to Windows-based development teams. | Last updated: May 26, 2026

1. Executive Summary

The team already uses the **GitHub Copilot IDE plugin** (code suggestions inside VS Code / JetBrains, etc.). Enabling the **GitHub Copilot CLI** is a meaningfully different decision: the CLI is an **agent that can execute shell commands and modify files on the developer's workstation** — not just a code-suggestion assistant.

This guide covers:

- New security risks the CLI introduces beyond the IDE plugin
- Enterprise controls to put in place *before* rollout
- Windows-specific operational guidance
- A 60–90 minute developer training outline
- A starter acceptable-use policy

2. What Changes vs. the IDE Plugin

Capability	IDE Plugin	Copilot CLI
Suggests code	Yes	Yes
Executes shell commands	No	Yes
Creates / modifies / deletes files	Limited (you accept)	Yes (agent-driven)
Runs autonomously across multiple steps	No	Yes (autopilot mode)
Can call MCP tools / external services	Limited	Yes
Can commit and push code	No	Yes
Has its own permission / approval system	N/A	Yes

Bottom line: the CLI moves Copilot from a *suggestor* to an *agent with execution rights*. Your threat model needs to account for that.

3. Top Security Concerns

3.1 Arbitrary command execution on the workstation

The agent can run shell commands, touch the filesystem, and invoke tools. By default, Copilot CLI can access the current working directory, its subdirectories, and the system temp directory. Path permissions apply to shell commands, file operations (create, edit, view), and search tools.

3.2 The "YOLO" / auto-approve trap

■ **This is the single biggest risk to train developers on.**

The CLI provides flags to bypass the permission system entirely:

- `--yolo` or `--allow-all` — auto-approves all tool executions
- `/yolo` or `/allow-all` — same thing as slash commands during an interactive session

Developers will be tempted to use these because the approval prompts get annoying. **Policy must be: never on a workstation with corporate credentials, VPN access, or source code.** If autonomy is needed, run inside a VM, container, or WSL2.

3.3 Directory trust & indirect prompt injection

A malicious `README.md`, code comment, or issue file in a cloned repository can contain instructions that the agent reads as part of its context. This is **indirect prompt injection** and is a well-documented attack vector against coding agents.

GitHub's own guidance: only launch Copilot CLI from directories you trust. Do not launch it from directories with executable files of unknown provenance, or with sensitive data you don't want exposed or modified.

3.4 MCP server expansion

Copilot CLI supports **Model Context Protocol (MCP) servers** — additional tools the agent can call. Workspace-level MCP servers (defined in `.mcp.json`, `.vscode/mcp.json`, or `devcontainer.json`) only load after the user confirms folder trust.

Risk: developers wiring up unsanctioned third-party MCP servers expands the agent's capabilities — and your attack surface — without security review.

3.5 Insecure code generation (still applies)

Same risk as the IDE plugin: Copilot can produce syntactically correct but insecure code (hardcoded secrets, SQLi, weak input validation, etc.). The CLI **amplifies** this because the agent can also commit and push. Code review discipline must remain.

3.6 Secrets exposure

The agent reads files in scope. If developers launch the CLI from `~ / C:\Users\<name>\`, files like `.env`, `.aws/credentials`, `.kube/config`, or `.ssh/` become part of its context.

3.7 Organization policy must be enabled

If Copilot is provided through an organization plan, the **Copilot CLI policy must be explicitly enabled** in the organization's GitHub settings. Verify which orgs and teams should have access before enabling broadly.

4. Enterprise Controls Before Rollout

4.1 Org-level policy

- Confirm Copilot CLI org policy is enabled only for sanctioned teams
- Define which repositories are in-scope vs. off-limits
- Document approved MCP servers (start with: none, expand on review)

4.2 Permission flags & wrappers

Use the deny-list and allow-list flags to constrain the agent:

```
# Block destructive shell commands while auto-approving safer ones
copilot --allow-all-tools --deny-tool='shell(rm)' --deny-tool='shell(git push)'

# Granular allow-list (preferred)
copilot --allow-tool='read' --allow-tool='write' --add-dir='./src'

# Restrict to a specific tool list – everything else unavailable
copilot --available-tools='read,write,shell(npm)'
```

The **safer pattern** is the opposite of `--allow-all`: granular `--allow-tool` and `--add-dir`, with `--allow-all-*` flags reserved for sandboxed environments.

4.3 Autopilot defaults

Autopilot mode lets Copilot work autonomously across multiple steps. It works best with full permissions, but you can run it with limited permissions — in which case the agent will deny any tool requests that require approval.

Recommended default: autopilot disabled or limited-permissions only until developers have completed training.

4.4 Logging & observability

Session logs are written to:

- Linux/macOS: `~/.copilot/logs/`
- **Windows:** `%USERPROFILE%\copilot\logs\`

Forward these to your SIEM (Splunk, Sentinel, etc.) for autonomous-run review and incident response.

4.5 Endpoint coordination

- Coordinate with EDR / Defender team — agent-driven file churn can trigger alerts; tune rather than disable
- Decide on Defender exclusions (if any) up front
- Document the expected install path and Node.js version

5. Windows-Specific Operational Guidance

5.1 Install prerequisites

- **Node.js (LTS)** — required. Standardize version and install path through your software catalog (Winget, SCCM, or Intune) rather than letting developers grab Node from anywhere.
- **Copilot CLI** — installed as a global npm package.

5.2 Terminal choice

Recommend **Windows Terminal + PowerShell 7**. The CLI's approval dialogs run in alt-screen mode with scrollable diffs; the legacy console host renders these poorly.

5.3 WSL2 as a sandbox

For any developer who wants to use --yo!o-style autonomy, **push them to run Copilot CLI inside WSL2 (Ubuntu) or a Dev Container**, not directly on the Windows host.

Why: WSL2 contains the blast radius away from the Windows workstation's domain-joined profile, OneDrive sync, mapped drives, and Windows Credential Manager.

5.4 Working directory hygiene

Developers often launch terminals from `C:\Users\<name>\`, which contains OneDrive, Documents, browser profiles, etc. Train them to:

- 1 `cd` into a **specific project directory first**
- 2 **Then** launch `copilot`

Never launch the CLI from a user-profile root.

5.5 Credentials & PATH

- Anything the user can run, the agent can run. Cached `gh auth / git` credentials are inherited.
- Prefer **short-lived tokens** (PATs with expiry, GitHub App tokens) over long-lived credentials.
- PowerShell execution policy does **not** meaningfully block the agent — don't rely on it as a control.

5.6 Defender / EDR

- Expect agent activity to generate alerts initially
- Tune detection rules rather than blanket-excluding the Copilot process
- Monitor `%USERPROFILE%\copilot\logs\` alongside EDR telemetry

6. Developer Training Outline (60–90 min)

Aim for a single live session per team, recorded for new joiners.

#	Module	Time	Content
1	What's different from the plugin	10 min	Agent vs. suggestor; execution rights, file modification, git operations; quick demo of the CLI running a simple task
2	The approval flow	15 min	Live demo: approve once / approve for session / approve permanently / deny; where each persists; why 'approve permanently' is rarely right
3	Trust prompt & sandboxing	10 min	Demo opening the CLI in an untrusted directory; WSL2 / Dev Container / VM patterns for autonomy. Rule: never --yolo on the corporate laptop
4	Prompt injection demo	10 min	Drop a malicious README.md into a sample repo; show the agent reading and acting on instructions inside it; how to spot and report
5	MCP servers	10 min	What MCP servers are; why adding random ones is risky; the org's approval process for new MCP servers
6	Secrets hygiene	5 min	.env, .aws/credentials, .kube/config should never be in scope; .gitignore and directory-scoping with --add-dir
7	Code review still applies	5 min	The agent can produce vulnerable code and can commit it; PR review, SAST, and dependency scanning remain mandatory
8	Incident response	5 min	How to read %USERPROFILE%\copilot\logs\; revoke session tokens / PATs; who to call (security on-call, IR contact)
9	AUP sign-off	5 min	Developers acknowledge the acceptable use policy before being granted CLI access

7. Acceptable Use Policy (Starter Template)

Replace [Company] with your org name before publishing. Have legal / security review before formal adoption.

Scope: All developers granted access to GitHub Copilot CLI on [Company]-managed workstations.

Rules:

- 1 **Launch only from a specific, trusted project directory.** Never launch the CLI from your user-profile root (C:\Users\<name>\ or ~).
- 2 **No --yolo / --allow-all / /yolo on the corporate workstation.** If autonomous operation is needed, run inside WSL2, a Dev Container, or an approved VM.
- 3 **No new MCP servers without security review.** Use only the approved list maintained by the security team.
- 4 **No agent operations against production systems or production credentials.** Use dev/staging environments and short-lived tokens.
- 5 **All Copilot-generated code goes through normal PR review, SAST, and dependency scanning.** No exceptions.
- 6 **Report anything unexpected immediately** — unexplained file changes, unexpected network calls, suspicious agent behavior — to [security contact].
- 7 **Do not store secrets in scope.** .env, cloud credential files, and SSH keys must remain outside the agent's working directory.

Acknowledgement: *I have completed the Copilot CLI training and agree to the rules above.*

Name: _____ Date: _____

Signature: _____

8. Open Questions for Your Team

- Which teams get CLI access first? (Pilot group)
- Which repos are explicitly out-of-scope?
- Who owns the approved-MCP-server list?
- Where do %USERPROFILE%\ .copilot\logs\ go in the SIEM?
- What's the rollback plan if a security incident occurs?
- Is autopilot allowed at all in v1, or limited-permissions only?

9. References

- [GitHub Docs — About GitHub Copilot CLI](#)
- [GitHub Docs — Responsible use of GitHub Copilot CLI](#)
- [GitHub Docs — Configure Copilot CLI](#)
- [GitHub Docs — Best practices for GitHub Copilot CLI](#)
- [GitHub Docs — Allowing Copilot CLI to work autonomously](#)

Draft document — review before circulating externally. Last updated May 26, 2026.